

记忆殿堂 MindPalace 作业报告

一、程序功能介绍

“记忆殿堂”是一款基于 C++17 和 Qt 6 开发的桌面端闪卡复习工具。程序的核心目标不是单纯保存问答卡片，而是围绕“整理知识、开始复习、根据掌握情况调整复习计划、查看学习进度”形成一条完整的学习流程。用户可以把单词、概念、公式、课堂笔记等内容整理成卡片，通过间隔重复的方式降低遗忘率。

程序启动后，主界面左侧显示已有卡组，用户可以新建、删除、重置卡组，也可以进入卡片管理窗口对某个卡组中的卡片进行新增、编辑和删除。每张卡片包含正面和背面两部分：正面通常放问题、术语或提示，背面放答案、解释或推导过程。卡组 and 卡片数据都保存在本地 JSON 文件中，程序不依赖云端账号，适合个人长期积累。

复习流程是本项目最主要的功能。用户选择某个卡组后，程序会筛选出今天到期的卡片，并在中间区域展示卡片正面。用户先尝试回忆，再点击“显示答案”查看背面内容，最后根据掌握情况选择“忘记、困难、普通、熟悉”等评分。评分提交后，程序会调用 SM-2 间隔重复算法更新卡片的复习次数、间隔、难度系数和下一次复习日期。也就是说，掌握得越好的卡片，下次出现得越晚；掌握不稳的卡片会更快重新进入复习队列。

除了手动创建卡片，程序还支持两类导入方式。第一类是 `.in/.out` 文件导入，其中 `.in` 文件按行保存卡片正面，`.out` 文件按行保存对应背面，适合已有成对题库的批量录入。第二类是 AI 辅助导入。用户在“设置 > AI助手”中填写服务商、API 密钥、Base URL 和模型名称后，可以选择 Markdown 等文档，由程序读取文档内容并调用模型生成卡组草稿。生成结果不会立刻写入本地数据，而是先进入预览窗口，用户可以翻看、编辑、新增或删除卡片，确认后再正式创建卡组。

项目还加入了进度统计和学习反馈。右侧统计面板会展示当前卡组的总卡片数、累计复习次数、掌握率、今日已复习数量和待复习进度。程序会记录每日复习量，并用近七日趋势图展示最近一周的学习情况。签到功能用于记录每天是否打开并完成学习，日历窗口则按日期显示复习安排，同时支持用户手动把某一天设置为强制复习或休息日。

在使用体验上，程序做了不少桌面化设计。主窗口支持主题切换，包括经典风格和暗色风格；卡片预览窗口可以双击卡组打开，并支持卡片翻面查看；复习提醒功能会在当天第一次打开程序时提示还有哪些卡组需要复习；桌面宠物开启后会常驻桌面，用户可以通过它快速打开主程序或查看今日复习概况。这些功能让程序不只是一个数据管理工具，而更接近一个陪伴式的复习助手。

二、项目各模块与类设计细节

项目整体采用比较清晰的分层结构：`model` 负责数据对象，`service` 负责算法、存储和外部服务，`controller` 负责业务流程和模块调度，`view` 负责 Qt 界面展示。这样的划分使界面代码

和核心逻辑没有混在一起，后续如果要更换界面样式、增加导入方式或调整复习算法，都可以在相对明确的位置修改。

Model 数据模型层

`Card` 类表示一张闪卡，是复习算法操作的最小单位。它保存卡片唯一 ID、正面内容、背面内容，以及 SM-2 算法需要的状态字段，包括 `repetitions`、`interval`、`easeFactor`、`lastReviewed` 和 `nextReviewDate`。新建卡片时，程序会自动生成 UUID，并把复习状态初始化为今天到期。`isDue()` 用于判断卡片今天是否应该复习，`toJson()` 和 `fromJson()` 则负责在内存对象和 JSON 数据之间转换。

`Deck` 类表示一个卡组，内部保存卡组 ID、卡组名称和一组 `Card` 对象。卡片使用 `std::unique_ptr<Card>` 管理所有权，避免对象复制带来的状态混乱。`addCard()` 用于向卡组加入卡片，`getDueCount()` 统计今天到期的卡片数量，`getReviewedTodayCount()` 根据 `lastReviewed` 统计今天已经复习过的卡片数量。它还维护 `manualSchedule`，用于保存用户在日历中设置的强制复习或休息安排。和 `Card` 类一样，`Deck` 也提供 JSON 序列化和反序列化接口。

Service 服务层

`StorageManager` 是项目的数据存储服务，采用静态工具类形式设计，不需要实例化。它负责把 `Deck` 保存为 JSON 文件，也负责从 JSON 文件恢复卡组对象。保存时先写入临时文件，再把旧文件改为备份，最后用临时文件替换正式文件；如果替换失败，会尽量恢复旧文件，减少半写入导致的数据损坏。除卡组存储外，它还管理每日复习记录和签到记录，例如

`incrementDailyReviewCount()`、`getWeeklyReviewData()`、`markTodaySignedIn()` 和 `getMonthlyCheckInDates()`。

`SM2Engine` 封装了间隔重复算法。它的核心接口是 `calculate(Card* card, int quality)`，根据用户评分更新卡片状态。当评分低于 3 时，说明用户没有成功回忆，程序会重置连续复习次数，并安排明天再次复习；当评分达到 3 及以上时，程序会增加复习次数，根据 SM-2 公式调整难度系数，并计算新的复习间隔。难度系数设置了下限，避免卡片因为几次困难评分后间隔变得不合理。

`AiAssistantService` 负责 AI 助手相关逻辑。它用 `AiAssistantSettings` 保存服务商、API Key、Base URL 和模型名称，用 `AiImportResult` 保存生成出来的牌组名称、摘要、来源文件名和卡片列表。它提供配置读取保存、默认模型选择、配置校验、连接测试、文档文本抽取和调用模型生成卡组等功能。文档抽取部分会限制文件大小和提示词长度，并对文本进行清理，减少模型生成格式错误的概率。网络调用通过 Qt Network 完成，整体上把 AI 相关代码隔离在服务层，避免主界面直接处理 API 细节。

Controller 控制层

`DeckController` 负责卡组 and 卡片的业务管理。构造时会从 `data/decks` 目录加载所有 JSON 卡组文件，并在内存中维护 `std::vector<Deck>`。它提供创建卡组、从卡片列表创建卡组、删除卡组、重命名卡组、重置复习进度、添加卡片、删除卡片、更新卡片和 `.in/.out` 导入等接口。这个类在修改数据时非常注意内存状态和磁盘状态的一致性：例如添加卡片后如果保存失败，会把刚加入的卡片从内存中撤回；删除卡片时如果写盘失败，会把卡片放回原来的位置；重命名失败时也会恢复旧名称。数据变化后，它会发出 `signal_deckListChanged()`，通知上层刷新界面。

`ReviewController` 负责一次复习会话的状态机。它定义了三个状态：`QuestionState` 表示正在显示问题，`AnswerState` 表示已经显示答案，`FinishedState` 表示本次复习结束。开始复习时，`startReview()` 会根据卡片是否到期构建队列，也可以在强制复习时忽略日期限制；如果用户开启了随机复习，它还会打乱卡片顺序。用户翻面时调用 `showAnswer()`，评分时调用 `submitFeedback()`。评分成功后，它会调用 `SM2Engine` 修改当前卡片，并立即通过 `StorageManager` 保存到文件。如果保存失败，则回滚卡片状态，保证复习记录不会只改内存而没有落盘。

`AppController` 是整个程序的总调度类。它拥有 `DeckController`、`ReviewController` 和 `MainWindow`，在启动时完成控制器初始化、主窗口创建和信号连接。主窗口发出的用户请求不会直接操作数据，而是先到 `AppController`，再转发给对应控制器。例如用户点击开始复习时，`AppController` 会找到目标卡组、更新统计面板、计算今日复习基线，然后调用 `ReviewController::startReview()`；用户提交评分后，它会更新每日复习日志和七日趋势图。它还负责复习提醒、日历数据查询、日历手动计划保存、签到状态同步等跨模块逻辑。

View 界面层

`MainWindow` 是主窗口类，也是用户操作最集中的界面。它继承自 `QMainWindow`，内部搭建左侧卡组列表、中间闪卡复习区、右侧统计区和菜单栏。它对外提供一组被动渲染接口，如 `updateDeckListView()`、`renderQuestionLayout()`、`renderAnswerLayout()`、`showFinishedSummaryPage()`、`updateProgressView()`、`updateSummaryStats()` 和 `updateWeeklyChart()`。同时，它通过 Qt 信号把用户意图抛给控制层，例如请求开始复习、提交评分、创建卡组、管理卡片、导入卡组、打开日历和签到等。

`CardManagerDialog` 是卡片管理对话框。它展示某个卡组中的全部卡片，并提供新增、删除和编辑入口。这个对话框本身不直接写文件，而是通过 `signal_requestAddCard`、`signal_requestDeleteCard` 和 `signal_requestUpdateCard` 把操作交给上层处理，符合 View 只负责交互和显示的设计。

`DeckPreviewDialog` 用于卡组预览，有普通学习预览和 AI 导入预览两种模式。普通模式主要用于双击已有卡组后浏览卡片，AI 导入模式则用于正式创建前检查生成结果。预览窗口中的卡片支持翻面、选择、编辑、新增和删除，用户确认后才会把 AI 生成的草稿转为真正的卡组。

`ScheduleCalendarDialog` 和 `CustomCalendarWidget` 负责复习日历。`CustomCalendarWidget` 重写 `paintCell()`，在日期格子里绘制复习安排和签到标记；`ScheduleCalendarDialog` 则负

责窗口布局、右键菜单和数据刷新。它通过信号向 `AppController` 请求某个月的复习安排和签到日期，也会把用户设置的强制复习或休息日发回控制层保存。

`WeeklyChartWidget` 是右侧近七日趋势图组件。它继承自 `QWidget`，通过 `setData()` 接收七天复习量和日期标签，并在 `paintEvent()` 中自绘图表。内部保存旧数据和动画进度，用于让统计图更新时有更自然的过渡效果。

`DesktopPetWidget` 是桌面宠物窗口。它负责绘制宠物贴图、处理拖拽、缩放、右键菜单和位置调整。相关命名空间 `MindPalace::DesktopPet` 还提供了是否启用、同步开机启动、检测主程序是否运行、启动进程等辅助函数。桌宠和主程序之间没有复杂的数据绑定，而是通过本地状态和进程启动逻辑提供轻量入口。

`ThemeRegistry`、`StyleUtils` 和 `StyledDialogs` 属于界面辅助模块。`ThemeRegistry` 统一登记可用主题，每个主题包含稳定 key、显示名称、QSS 路径和是否使用磨砂卡片效果等信息；`StyleUtils` 封装常用样式操作，如背景、文字颜色、按钮字体和阴影；`StyledDialogs` 统一弹窗样式，减少不同窗口之间的视觉割裂。

模块协作流程

以一次正常复习为例，用户在 `MainWindow` 中选择卡组并请求开始复习，信号传到 `AppController`。`AppController` 从 `DeckController` 找到对应 `Deck`，再把它交给 `ReviewController`。`ReviewController` 筛选到期卡片并发出显示问题的信号，`AppController` 接收后调用 `MainWindow` 渲染正面。用户翻面后，`ReviewController` 切换到答案态；用户评分后，`ReviewController` 调用 `SM2Engine` 更新卡片，并通过 `StorageManager` 保存到 JSON 文件。保存成功后，队列进入下一张卡片或结束本次复习，界面同步刷新进度和统计。

这种设计的优点是职责比较明确：界面只描述用户做了什么，控制器决定接下来该调用谁，服务层处理具体算法和存储，模型层只保存真实数据。项目功能虽然包括复习、导入、统计、日历、签到、桌宠和主题等多个方向，但通过分层后，每个类的边界仍然比较清楚，后续维护和扩展都相对方便。

三、分工情况

本项目全程使用GitHub进行版本控制与协作开发，每位成员在GitHub上创建分支进行开发，通过 Pull Request 的方式将代码合并到主分支。成员间分工合理，协作积极，具体分工如下，具体分工也可查看GitHub提交记录：

- 易瑞丰：构思并确定项目功能与实现思路；搭建代码的MVC框架，分为模型层，视图层与控制层三部分；主要负责编写模型层代码，定义了Card、Deck等核心数据结构，同时参与视图层美化。
- 张昊洋：主要负责控制层的编写，实现了闪卡调度的核心SM-2算法、卡片的持久化存储等功能；负责项目介绍视频录制。

- 任子博：主要负责视图层的编写，设计用户交互界面与闪卡动效，同时参与控制层编写，实现了卡片的增删改查等功能；推动项目采用PR进行规范化开发，设置GitHub CI进行自动化测试。

四、总结与反思

MindPalace是一个用户友好，功能强大的闪卡学习工具，集成了复习日历、自动学习统计、智能调度算法等功能。项目采用MVC架构设计，前后端分离，代码结构清晰，易于维护与扩展。用户只需要打开程序，就可以直接开始沉浸式复习，操作十分简便。

我们团队在开发过程中全程使用git进行版本控制，充分利用分支开发与PR合并的方式进行协作，代码必须经过审查才能合并到主分支。这种方式使得团队成员在审查代码时便能充分了解其他成员的工作与项目进度，并且及时发现问题进行交流。通过这种方式，我们团队的协作效率得到了显著提升，也及时避免了许多bug，例如前后端对Card的数据约定不同等。

我们小组中每位成员此前都没有使用Qt进行项目开发，但通过查阅Qt官方文档、观看视频教程以及寻求AI辅助的方式，我们掌握了Qt的基本语法，控件使用方法以及信号与槽机制等知识。这次项目开发经历也让组员对面向对象的设计有了更深刻的理解，比如对象如何封装，如何保证可扩展性，如何解耦不同控制层之间的逻辑等等，收获颇丰。此外，作为来自心理（易瑞丰）、数学（任子博）与计算机（张昊洋）三个不同院系的小组，我们在项目中感受到了不同学科的交叉融合，扩展了知识面。

由于时间和精力有限，我们的项目仍然有不足之处。例如当前不支持卡片中插入图像，同时统计功能有待完善，可以加入学习曲线等可视化功能。如果更进一步，还可以考虑支持多平台以及云端同步等功能。对于卡片操作，也可以考虑加入批量操作与搜索功能，提升用户体验。

总的来说，Mind Palace是一次将 Qt 技术、C++ 编程、认知心理学算法和软件工程设计结合起来的综合实践。通过本项目，我们不仅完成了一个具有实际意义的学习工具，也在团队协作、架构设计、问题排查和用户体验思维方面获得了很大提升，希望未来能够继续完善这个项目，让它能够真正帮助同学们的记忆。